

MADES Programming Challenge 25-26 - Editorial

Marmara University Developer Society - Competitive Programming Club

May 2026

A. Gas Money

To find the minimum number of trips, it is optimal to take as many people as possible on each trip. To get the answer, we just need to divide x by 5 and round up.

To safely round up using integer math without relying on floating-point functions like `ceil()`, we can add 4 (which is $max_capacity - 1$) to x before dividing. Formula: $(x + 4)/5$.

Code Implementation (C++)

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x;
6     cin >> x;
7     cout << (x + 4) / 5 << "\n";
8     return 0;
9 }
```

Code Implementation (Python 3)

```
1 x = int(input())
2 print((x + 4) // 5)
```

Code Implementation (Java)

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int x = sc.nextInt();
7         System.out.println((x + 4) / 5);
8     }
9 }
```

Code Implementation (C#)

```
1 using System;
2
3 class Program {
4     static void Main() {
5         int x = int.Parse(Console.ReadLine());
6         Console.WriteLine((x + 4) / 5);
7     }
8 }
```

B. Free Coffee

The key is to look at multiples of 3. Why?

- If n is a multiple of 3 ($n \% 3 == 0$), no matter what Alice does, she will leave a number that is not divisible by 3, then bob can take the exact amount needed to bring the total back to a multiple of 3 (if she takes 1 bob takes 2 and vice versa), that way he is guaranteed to hit 0.
- If n is not a multiple of 3, Alice can just take the remainder ($n \% 3$). This leaves a multiple of 3 for Bob, effectively stealing the winning strategy and making Alice the guaranteed winner.

Another way to look at the problem is through game states. The idea is that the only way for a player to be "guaranteed" a win is to be able to give their opponent a position that is "guaranteed" to lose.

At the beginning, the only losing number is 0, and following that logic, that makes both 1 and 2 winning numbers, and 3 a losing number. If you go on applying the same recursive logic the following pattern emerges: Only multiples of 3 are losing here.

Code Implementation (C++)

```
1 #include <iostream>
2 using namespace std;
3
4 void solve() {
5     int n;
6     cin >> n;
7     if (n % 3 == 0) cout << "Bob\n";
8     else cout << "Alice\n";
9 }
10
11 int main() {
12     int t; cin >> t;
13     while (t--) solve();
14     return 0;
15 }
```

Code Implementation (Python 3)

```
1 t = int(input())
2 for _ in range(t):
3     n = int(input())
4     if n % 3 == 0:
5         print("Bob")
6     else:
7         print("Alice")
```

Code Implementation (Java)

```
1 import java.util.Scanner;
2
3 public class Main {
```

```

4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int t = sc.nextInt();
7         while (t-- > 0) {
8             int n = sc.nextLong();
9             if (n % 3 == 0) System.out.println("Bob");
10            else System.out.println("Alice");
11        }
12    }
13 }

```

Code Implementation (C#)

```

1 using System;
2
3 class Program {
4     static void Main() {
5         int t = int.Parse(Console.ReadLine());
6         while (t-- > 0) {
7             int n = int.Parse(Console.ReadLine());
8             if (n % 3 == 0) Console.WriteLine("Bob");
9             else Console.WriteLine("Alice");
10        }
11    }
12 }

```

C. Broken Keyboard

We need to simulate Bob's typing. We can use a dynamic array (or a String Builder) as our typed screen. We maintain two boolean variables: `caps` and `shift`.

When iterating through the string:

- If we see `\`, we delete the last character from our screen (if any exist). We also consume the shift state (`shift = false`).
- If we see `#`, we toggle caps lock (`caps = !caps`) and consume shift.
- If we see `*`, we make `shift = true`
- For any regular letter, we determine if it should be uppercase. It will be uppercase if exactly one of `caps` or `shift` is active (an XOR operation: `caps ^ shift`). After typing, we consume the shift state.

The most common bugs I've seen for this problem are that people fail to handle either of the following:

1. the possibility that a `\` could be typed when there are no letters currently on screen, which often results in a compilation error.
2. **ANY** keystroke after a `*` consumes it, so for example `*#a` would be `A`.

so please make sure to read the problem statement carefully.

Code Implementation (C++)

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 void solve() {
6     string s;
7     cin >> s;
8     string res = "";
9     bool caps = false, shift = false;
10
11     for (char c : s) {
12         if (c == '\\') {
13             if (!res.empty()) res.pop_back();
14             shift = false;
15         } else if (c == '#') {
16             caps = !caps;
17             shift = false;
18         } else if (c == '*') {
19             shift = true;
20         } else {
21             bool is_upper = caps ^ shift;
22             if (is_upper) res += toupper(c);
23             else res += tolower(c);
24             shift = false;
25         }
26     }
27     cout << res << "\n";
28 }
29
```

```

30 int main() {
31     int t; cin >> t;
32     while (t--> 0) solve();
33     return 0;
34 }

```

Code Implementation (Python 3)

```

1 t = int(input())
2 for _ in range(t):
3     s = input()
4     res = []
5     caps, shift = False, False
6
7     for c in s:
8         if c == '\\':
9             if res:
10                 res.pop()
11                 shift = False
12             elif c == '#':
13                 caps = not caps
14                 shift = False
15             elif c == '*':
16                 shift = True
17             else:
18                 if caps ^ shift:
19                     res.append(c.upper())
20                 else:
21                     res.append(c.lower())
22                 shift = False
23
24     print("".join(res))

```

Code Implementation (Java)

```

1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int t = sc.nextInt();
7         while (t-- > 0) {
8             String s = sc.next();
9             StringBuilder res = new StringBuilder();
10            boolean caps = false, shift = false;
11
12            for (char c : s.toCharArray()) {
13                if (c == '\\') {
14                    if (res.length() > 0) res.deleteCharAt(res.length()
15                        - 1);
16                    shift = false;
17                } else if (c == '#') {
18                    caps = !caps;
19                    shift = false;
20                } else if (c == '*') {
21                    shift = true;

```

```

21         } else {
22             boolean isUpper = caps ^ shift;
23             res.append(isUpper ? Character.toUpperCase(c) :
Character.toLowerCase(c));
24             shift = false;
25         }
26     }
27     System.out.println(res.toString());
28 }
29 }
30 }

```

Code Implementation (C#)

```

1  using System;
2  using System.Text;
3
4  class Program {
5      static void Main() {
6          int t = int.Parse(Console.ReadLine());
7          while (t-- > 0) {
8              string s = Console.ReadLine();
9              StringBuilder res = new StringBuilder();
10             bool caps = false, shift = false;
11
12             foreach (char c in s) {
13                 if (c == '\\') {
14                     if (res.Length > 0) res.Length--;
15                     shift = false;
16                 } else if (c == '#') {
17                     caps = !caps;
18                     shift = false;
19                 } else if (c == '*') {
20                     shift = true;
21                 } else {
22                     bool isUpper = caps ^ shift;
23                     res.Append(isUpper ? char.ToUpper(c) : char.ToLower
(c));
24                     shift = false;
25                 }
26             }
27             Console.WriteLine(res.ToString());
28         }
29     }
30 }

```

D. Hidden Treasure

While it seems complex, the bounds of the problem make it incredibly simple. The grid is bounded from -100 to 100 on both axes. This means there are only $201 \times 201 = 40,401$ possible locations for the treasure!

We can then check the validity of each location by looping through our n anchors bringing our worst case complexity to $201 \times 201 \times 1000 \approx 4 * 10^7$ operations which comfortably passes in one second.

There is also a more optimal $O(n)$ solution where, instead of checking every single grid cell, we can optimize: Look at the very first anchor point. Given its Manhattan distance d_0 , the treasure *must* lie somewhere exactly on the perimeter of a diamond shape around this point. This, however, is a bit overkill for the given constraints so we won't go into it much.

Code Implementation (C++)

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 using namespace std;
5
6 struct Anchor { int x, y, d; };
7
8 int main() {
9     int n; cin >> n;
10    vector<Anchor> a(n);
11    for (int i = 0; i < n; i++) cin >> a[i].x >> a[i].y >> a[i].d;
12
13    for (int x = -100; x <= 100; x++) {
14        for (int y = -100; y <= 100; y++) {
15            bool valid = true;
16
17            for (int i = 0; i < n; i++) {
18                if (abs(x - a[i].x) + abs(y - a[i].y) != a[i].d) {
19                    valid = false;
20                    break;
21                }
22            }
23
24            if (valid) {
25                cout << x << " " << y << "\n";
26                return 0;
27            }
28        }
29    }
30    cout << "-1\n";
31    return 0;
32 }
```

Code Implementation (Python 3)

```
1 n = int(input())
2 anchors = [tuple(map(int, input().split())) for _ in range(n)]
3
4 found = False
```

```

5 for x in range(-100, 101):
6     for y in range(-100, 101):
7         valid = True
8
9         for xi, yi, di in anchors:
10             if abs(x - xi) + abs(y - yi) != di:
11                 valid = False
12                 break
13
14         if valid:
15             print(f"{x} {y}")
16             found = True
17             break
18
19     if found:
20         break
21
22 if not found:
23     print("-1")

```

Code Implementation (Java)

```

1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int n = sc.nextInt();
7         int[] ax = new int[n], ay = new int[n], ad = new int[n];
8
9         for (int i = 0; i < n; i++) {
10             ax[i] = sc.nextInt();
11             ay[i] = sc.nextInt();
12             ad[i] = sc.nextInt();
13         }
14
15
16         for (int x = -100; x <= 100; x++) {
17             for (int y = -100; y <= 100; y++) {
18                 boolean valid = true;
19
20
21                 for (int i = 0; i < n; i++) {
22                     if (Math.abs(x - ax[i]) + Math.abs(y - ay[i]) != ad
23 [i]) {
24                         valid = false;
25                         break;
26                     }
27                 }
28
29                 if (valid) {
30                     System.out.println(x + " " + y);
31                     return;
32                 }
33             }
34             System.out.println("-1");

```

```

35     }
36 }

```

Code Implementation (C#)

```

1  using System;
2
3  class Program {
4      static void Main() {
5          int n = int.Parse(Console.ReadLine());
6          int[] ax = new int[n], ay = new int[n], ad = new int[n];
7
8          for (int i = 0; i < n; i++) {
9              string[] parts = Console.ReadLine().Split();
10             ax[i] = int.Parse(parts[0]);
11             ay[i] = int.Parse(parts[1]);
12             ad[i] = int.Parse(parts[2]);
13         }
14
15         for (int x = -100; x <= 100; x++) {
16             for (int y = -100; y <= 100; y++) {
17                 bool valid = true;
18
19                 for (int i = 0; i < n; i++) {
20                     if (Math.Abs(x - ax[i]) + Math.Abs(y - ay[i]) != ad
[i]) {
21                         valid = false;
22                         break;
23                     }
24                 }
25
26                 if (valid) {
27                     Console.WriteLine($"{x} {y}");
28                     return;
29                 }
30             }
31         }
32         Console.WriteLine("-1");
33     }
34 }

```

E. To The Moon!

If we process each query by counting days via a loop from l to r , we will get a Time Limit Exceeded (TLE) verdict. This is because $q \leq 10^5$ and $n \leq 10^5$, leading to potentially 10^{10} operations!

We need $O(1)$ query resolution. This is the perfect use-case for a **Prefix Sum Array**. Let's build an array G where $G[i] = 1$ if day i is a Green Day, and 0 otherwise. We then transform this into a prefix sum array P , where $P[i] = P[i - 1] + G[i]$.

For any query asking for Green Days strictly within the window l to r , the answer is simply $P[r] - P[l]$.

Code Implementation (C++)

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     int n, q;
7     cin >> n >> q;
8
9     vector<int> a(n + 1);
10    for (int i = 1; i <= n; i++) cin >> a[i];
11
12    vector<int> P(n + 1, 0);
13    for (int i = 2; i <= n; i++) {
14        P[i] = P[i - 1] + (a[i] > a[i - 1] ? 1 : 0);
15    }
16
17    while (q--) {
18        int l, r;
19        cin >> l >> r;
20        if (l == r) cout << "0\n";
21        else cout << P[r] - P[l] << "\n";
22    }
23    return 0;
24 }
```

Code Implementation (Python 3)

```
1 n, q = map(int, input().split())
2
3 a = [0] + list(map(int, input().split()))
4
5 P = [0] * (n + 1)
6 for i in range(2, n + 1):
7     P[i] = P[i - 1] + (1 if a[i] > a[i - 1] else 0)
8
9
10 for _ in range(q):
11     l, r = map(int, input().split())
12
13     if l == r:
14         print("0")
```

```

15     else:
16         print(P[r] - P[l])

```

Code Implementation (Java)

```

1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          int n = sc.nextInt();
7          int q = sc.nextInt();
8
9          int[] a = new int[n + 1];
10         for (int i = 1; i <= n; i++) {
11             a[i] = sc.nextInt();
12         }
13
14         int[] P = new int[n + 1];
15         for (int i = 2; i <= n; i++) {
16             P[i] = P[i - 1] + (a[i] > a[i - 1] ? 1 : 0);
17         }
18
19         for (int i = 0; i < q; i++) {
20             int l = sc.nextInt();
21             int r = sc.nextInt();
22             if (l == r) System.out.println(0);
23             else System.out.println(P[r] - P[l]);
24         }
25     }
26 }
27 \end{Java}

```

Code Implementation (C#)

```

1  using System;
2
3  class Program {
4      static void Main() {
5          string[] nq = Console.ReadLine().Split();
6          int n = int.Parse(nq[0]);
7          int q = int.Parse(nq[1]);
8
9          string[] arr = Console.ReadLine().Split();
10         int[] a = new int[n + 1];
11         for (int i = 1; i <= n; i++) {
12             a[i] = int.Parse(arr[i - 1]);
13         }
14
15         int[] P = new int[n + 1];
16         for (int i = 2; i <= n; i++) {
17             P[i] = P[i - 1] + (a[i] > a[i - 1] ? 1 : 0);
18         }
19
20         for (int i = 0; i < q; i++) {
21             string[] lr = Console.ReadLine().Split();

```

```
22         int l = int.Parse(lr[0]);
23         int r = int.Parse(lr[1]);
24
25         if (l == r) Console.WriteLine(0);
26         else Console.WriteLine(P[r] - P[l]);
27     }
28 }
29 }
```

F. Algorithmus and the Final Project

To minimize the number of teams, we should group students with the most similar Skill Ratings together.

This naturally calls for a **Greedy** approach:

1. Sort the array of skill ratings in ascending order.
2. Start the first team with the lowest rated student.
3. Keep iterating through the students. As long as the current student's skill rating minus the team's lowest skill rating is $\leq k$, they can safely join the current team.
4. The moment the difference exceeds k , we are forced to close the current team and start a brand new team anchored by this current student.

Because the array is sorted, this greedy extension ensures each team takes up the maximum possible number of valid members before starting a new group.

What is "Greedy" about this approach is that we choose the option that looks the best **right now** without really looking at the bigger picture, which, apparently, is sometimes optimal.

Code Implementation (C++)

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      int n, k;
8      cin >> n >> k;
9
10     vector<int> a(n);
11     for (int i = 0; i < n; i++) cin >> a[i];
12
13     sort(a.begin(), a.end());
14
15     int teams = 1;
16     int current_min = a[0];
17
18     for (int i = 1; i < n; i++) {
19         if (a[i] - current_min > k) {
20             teams++;
21             current_min = a[i];
22         }
23     }
24
25     cout << teams << "\n";
26     return 0;
27 }
```

Code Implementation (Python 3)

```
1 n, k = map(int, input().split())
2 a = list(map(int, input().split()))
3
4 a.sort()
5
6 teams = 1
7 current_min = a[0]
8
9 for i in range(1, n):
10     if a[i] - current_min > k:
11         teams += 1
12         current_min = a[i]
13
14 print(teams)
```

Code Implementation (Java)

```
1 import java.util.Scanner;
2 import java.util.Arrays;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         int n = sc.nextInt();
8         int k = sc.nextInt();
9
10        int[] a = new int[n];
11        for (int i = 0; i < n; i++) {
12            a[i] = sc.nextInt();
13        }
14
15        Arrays.sort(a);
16
17        int teams = 1;
18        int currentMin = a[0];
19
20        for (int i = 1; i < n; i++) {
21            if (a[i] - currentMin > k) {
22                teams++;
23                currentMin = a[i];
24            }
25        }
26
27        System.out.println(teams);
28    }
29 }
```

Code Implementation (C#)

```
1 using System;
2
3 class Program {
4     static void Main() {
5         string[] nk = Console.ReadLine().Split();
```

```

6      int n = int.Parse(nk[0]);
7      int k = int.Parse(nk[1]);
8
9      string[] arr = Console.ReadLine().Split();
10     int[] a = new int[n];
11     for (int i = 0; i < n; i++) {
12         a[i] = int.Parse(arr[i]);
13     }
14
15     Array.Sort(a);
16
17     int teams = 1;
18     int currentMin = a[0];
19
20     for (int i = 1; i < n; i++) {
21         if (a[i] - currentMin > k) {
22             teams++;
23             currentMin = a[i];
24         }
25     }
26
27     Console.WriteLine(teams);
28 }
29 }

```